



Documentación Técnica Arquitectónica

Proyecto NODO: Plataforma Cognitiva Modular v0.1

Martín Alejandro Márquez Pínto
Ingeniería Robótica (3er Semestre) - Universidad de Guadalajara

14 de Febrero de 2026

Resumen

El presente documento detalla la arquitectura fundacional de NODO (v0.1), una plataforma robótica escalable diseñada mediante *Clean Architecture* y Programación Orientada a Objetos en C++. El proyecto establece las bases operativas de hardware y software para evolucionar progresivamente desde un sistema reactivo básico hasta una entidad con procesamiento cognitivo multimodal y aprendizaje automático.

Índice

1. Introducción	2
2. Especificaciones de Hardware (v0.1)	2
2.1. Esquemático de Conexiones	2
3. Arquitectura de Software	2
3.1. Reglas Críticas de Desarrollo	2
4. Implementación del Código Fuente	3
4.1. Configuración del Entorno (<code>platformio.ini</code>)	3
4.2. Módulo de Entrada: <code>ProcesadorAudio</code>	3
4.3. Módulo Central: <code>MaquinaEstados</code>	4
4.4. Módulo de Salida: <code>GestorPantalla</code>	5
4.5. Archivo Principal: <code>main.cpp</code>	6
5. Hoja de Ruta (Roadmap)	7
6. Anexo: Prompt Maestro de Continuidad AI	7



1. Introducción

El proyecto NODO nace bajo la visión de desarrollar un asistente/acompañante tecnológico de uso personal. Al ser un proyecto a largo plazo proyectado para escalar durante la formación académica en ingeniería, se estableció como requisito indispensable abandonar las prácticas de programación de aficionados (código monolítico bloqueante) e implementar estándares de la industria desde el día uno.

El paradigma seleccionado es el modelo **IPO** (Input, Process, Output), implementado mediante un microcontrolador que delega responsabilidades a módulos independientes. El objetivo de la versión 0.1 es lograr un *Minimum Viable Product* (MVP) capaz de percibir su entorno (audio), procesar la información lógicamente (máquina de estados) y emitir una respuesta visual (interfaz OLED).

2. Especificaciones de Hardware (v0.1)

El hardware base fue seleccionado por su accesibilidad y documentación técnica, ideal para la fase de prototipado temprano (*Cerebro Reptiliano*).

- **Microcontrolador Base:** Arduino UNO (ATmega328P).
- **Sensor Auditivo (Input):** Módulo de micrófono amplificador operacional **MAX4466**. A diferencia de sensores digitales comerciales, este entrega una onda analógica real. Para evitar ruido electromagnético de la línea USB (5V), se alimenta mediante el regulador interno de 3,3V del Arduino.
- **Interfaz Visual (Output):** Pantalla OLED de 0.96 pulgadas con controlador **SSD1306**. La comunicación se realiza mediante el protocolo I2C, optimizando el uso de pines del microcontrolador.

2.1. Esquemático de Conexiones

- **MAX4466:** VCC → 3.3V | GND → GND | OUT → A0
- **OLED SSD1306:** VCC → 5V | GND → GND | SDA → A4 | SCL → A5

3. Arquitectura de Software

El entorno de desarrollo utilizado es **Visual Studio Code** con la extensión **PlatformIO**. Esta decisión permite abandonar el entorno clásico de Arduino y abrazar el uso de gestores de dependencias y modularización profunda.

3.1. Reglas Críticas de Desarrollo

1. **Prohibición de retrasos bloqueantes:** Está estrictamente prohibido el uso de la función `delay()`. Todo evento temporal debe gestionarse mediante contadores diferenciales usando la función `millis()`.
2. **Arquitectura Limpia:** El archivo principal (`main.cpp`) actúa únicamente como director de orquesta. La lógica matemática, procesamiento de señales y rutinas gráficas están encapsuladas en librerías privadas en C++ dentro del directorio `/lib`.
3. **DSP (Digital Signal Processing):** Para la lectura analógica, se implementó un algoritmo de muestreo por ventana de tiempo (50 ms) que calcula la amplitud pico a pico, fungiendo como un filtro de ruido ambiental para evitar falsos positivos.



4. Implementación del Código Fuente

A continuación, se detalla el código base de la versión 0.1, el cual está respaldado bajo control de versiones en GitHub.

4.1. Configuración del Entorno (platformio.ini)

El archivo de inicialización gestiona automáticamente la descarga de librerías y configura la velocidad del Monitor Serial para evitar cuellos de botella en la lectura de audio.

```
1 [env:uno]
2 platform = atmelavr
3 board = uno
4 framework = arduino
5 monitor_speed = 115200
6 lib_deps =
7   adafruit/Adafruit GFX Library @ ~1.11.9
8   adafruit/Adafruit SSD1306 @ ~2.5.9
```

Listing 1: Configuración en platformio.ini

4.2. Módulo de Entrada: ProcesadorAudio

Este módulo encapsula la lectura de hardware y las matemáticas necesarias para transformar el voltaje analógico en una variable booleana de evento (ruido fuerte detectado).

```
1 #ifndef PROCESADOR_AUDIO_H
2 #define PROCESADOR_AUDIO_H
3 #include <Arduino.h>
4
5 class ProcesadorAudio {
6 private:
7     int pinMicrofono;
8     int umbralDeteccion;
9     int ventanaTiempo;
10
11 public:
12     ProcesadorAudio(int pin, int umbral);
13     void inicializar();
14     int obtenerAmplitud();
15     bool detectarEventoFuerte();
16 };
17 #endif
```

Listing 2: lib/ProcesadorAudio/ProcesadorAudio.h

```
1 #include "ProcesadorAudio.h"
2
3 ProcesadorAudio::ProcesadorAudio(int pin, int umbral) {
4     pinMicrofono = pin;
5     umbralDeteccion = umbral;
6     ventanaTiempo = 50; // Muestreo de 50 ms
7 }
8
9 void ProcesadorAudio::inicializar() {
10     pinMode(pinMicrofono, INPUT);
11 }
12
13 int ProcesadorAudio::obtenerAmplitud() {
14     unsigned long inicioMillis = millis();
15     int valorMaximo = 0;
16     int valorMinimo = 1024;
17
18     while (millis() - inicioMillis < (unsigned long)ventanaTiempo) {
19         int lectura = analogRead(pinMicrofono);
```



```
20         if (lectura > valorMaximo) valorMaximo = lectura;
21         if (lectura < valorMinimo) valorMinimo = lectura;
22     }
23     return (valorMaximo - valorMinimo);
24 }
25
26 bool ProcesadorAudio::detectarEventoFuerte() {
27     int amplitudActual = obtenerAmplitud();
28     return (amplitudActual > umbralDeteccion);
29 }
```

Listing 3: lib/ProcesadorAudio/ProcesadorAudio.cpp

4.3. Módulo Central: MaquinaEstados

El cerebro del sistema. Utiliza una Máquina de Estados Finitos (FSM) basada en temporizadores de hardware interno, lo que garantiza que el sistema permanezca reactivo sin importar en qué estado se encuentre.

```
1 #ifndef MAQUINA_ESTADOS_H
2 #define MAQUINA_ESTADOS_H
3 #include <Arduino.h>
4
5 enum EstadoNodo {
6     DURMIENDO,
7     FELIZ
8 };
9
10 class MaquinaEstados {
11     private:
12         EstadoNodo estadoActual;
13         unsigned long tiempoUltimoCambio;
14         unsigned long duracionFeliz;
15
16     public:
17         MaquinaEstados();
18         void inicializar();
19         void procesarEventoAudio(bool escuchoFuerte);
20         void actualizar();
21         EstadoNodo obtenerEstadoActual();
22 };
23 #endif
```

Listing 4: lib/MaquinaEstados/MaquinaEstados.h

```
1 #include "MaquinaEstados.h"
2
3 MaquinaEstados::MaquinaEstados() {
4     estadoActual = DURMIENDO;
5     tiempoUltimoCambio = 0;
6     duracionFeliz = 3000; // 3 segundos de duracion
7 }
8
9 void MaquinaEstados::inicializar() {
10     estadoActual = DURMIENDO;
11 }
12
13 void MaquinaEstados::procesarEventoAudio(bool escuchoFuerte) {
14     if (escuchoFuerte && estadoActual == DURMIENDO) {
15         estadoActual = FELIZ;
16         tiempoUltimoCambio = millis();
17     }
18 }
19
20 void MaquinaEstados::actualizar() {
21     if (estadoActual == FELIZ) {
22         if (millis() - tiempoUltimoCambio >= duracionFeliz) {
23             estadoActual = DURMIENDO;
```



```
24     }
25 }
26 }
27
28 EstadoNodo MaquinaEstados::obtenerEstadoActual() {
29     return estadoActual;
30 }
```

Listing 5: lib/MaquinaEstados/MaquinaEstados.cpp

4.4. Módulo de Salida: GestorPantalla

Controla la Interfaz de Usuario. Incorpora una lógica *Anti-Flicker* para evitar el parpadeo enviando comandos I2C a la pantalla única y exclusivamente cuando el módulo `MaquinaEstados` reporta un cambio de actitud.

```
1 #ifndef GESTOR_PANTALLA_H
2 #define GESTOR_PANTALLA_H
3 #include <Arduino.h>
4 #include <Adafruit_GFX.h>
5 #include <Adafruit_SSD1306.h>
6 #include <MaquinaEstados.h>
7
8 class GestorPantalla {
9     private:
10         Adafruit_SSD1306 display;
11         EstadoNodo estadoAnterior;
12         void dibujarCaraDurmiendo();
13         void dibujarCaraFeliz();
14
15     public:
16         GestorPantalla();
17         void inicializar();
18         void actualizar(EstadoNodo estadoActual);
19 };
20 #endif
```

Listing 6: lib/GestorPantalla/GestorPantalla.h

```
1 #include "GestorPantalla.h"
2
3 GestorPantalla::GestorPantalla() : display(128, 64, &Wire, -1) {
4     estadoAnterior = (EstadoNodo)-1;
5 }
6
7 void GestorPantalla::inicializar() {
8     if(!display.begin(SSD1306_SWITCHCAPVCC, 0x3C)) {
9         for(;;); // Bucle infinito si falla
10    }
11    display.clearDisplay();
12    display.setTextSize(1);
13    display.setTextColor(SSD1306_WHITE);
14    display.setCursor(20, 30);
15    display.println("Iniciando NODO...");
16    display.display();
17    delay(1000);
18 }
19
20 void GestorPantalla::actualizar(EstadoNodo estadoActual) {
21     if (estadoActual != estadoAnterior) {
22         display.clearDisplay();
23         if (estadoActual == DURMIENDO) {
24             dibujarCaraDurmiendo();
25         } else if (estadoActual == FELIZ) {
26             dibujarCaraFeliz();
27         }
28         display.display();
29         estadoAnterior = estadoActual;
30     }
```



```
30     }
31 }
32
33 void GestorPantalla::dibujarCaraDurmiendo() {
34     display.drawLine(30, 32, 50, 32, SSD1306_WHITE);
35     display.drawLine(78, 32, 98, 32, SSD1306_WHITE);
36     display.setCursor(100, 10);
37     display.print("z");
38 }
39
40 void GestorPantalla::dibujarCaraFeliz() {
41     display.fillCircle(40, 32, 10, SSD1306_WHITE);
42     display.fillCircle(88, 32, 10, SSD1306_WHITE);
43     display.drawPixel(64, 45, SSD1306_WHITE);
44     display.drawPixel(63, 44, SSD1306_WHITE);
45     display.drawPixel(65, 44, SSD1306_WHITE);
46 }
```

Listing 7: lib/GestorPantalla/GestorPantalla.cpp

4.5. Archivo Principal: main.cpp

El núcleo de ejecución del microcontrolador. Mantiene la limpieza arquitectónica delegando las tareas a los módulos correspondientes en un ciclo ininterrumpido.

```
1  // Nacimiento de NODO (Nombre Clave) el 14/02/2026 "Plataforma Cognitiva Modular" by Marquez
   // Pinto Martin Alejandro.
2
3  #include <Arduino.h>
4  #include <ProcesadorAudio.h>
5  #include <MaquinaEstados.h>
6  #include <GestorPantalla.h>
7
8  ProcesadorAudio oidoDeNodo(A0, 250);
9  MaquinaEstados cerebroDeNodo;
10 GestorPantalla rostroDeNodo;
11
12 void setup() {
13     Serial.begin(115200);
14     rostroDeNodo.inicializar();
15     oidoDeNodo.inicializar();
16     cerebroDeNodo.inicializar();
17 }
18
19 void loop() {
20     // 1. INPUT
21     bool huboAplauso = oidoDeNodo.detectarEventoFuerte();
22
23     // 2. PROCESS
24     cerebroDeNodo.procesarEventoAudio(huboAplauso);
25     cerebroDeNodo.actualizar();
26
27     // 3. OUTPUT
28     EstadoNodo humorActual = cerebroDeNodo.obtenerEstadoActual();
29     rostroDeNodo.actualizar(humorActual);
30 }
```

Listing 8: src/main.cpp



5. Hoja de Ruta (Roadmap)

El desarrollo de NODO seguirá una curva progresiva en las siguientes versiones iterativas:

- **v0.2 (Memoria Emocional):** Incorporación de un contador de estrés. Si el micrófono detecta múltiples eventos auditivos fuertes en una ventana de tiempo reducida (ej. 3 aplausos en 10 segundos), la Máquina de Estados transicionará a un nuevo estado **ENOJADO/ESTRESADO**.
- **v0.3 (Experiencia de Usuario Animada):** Implementación de algoritmos de números pseudoaleatorios (`random()`) para generar parpadeos esporádicos y variaciones en la pantalla OLED, sin usar demoras bloqueantes.
- **v0.4 (Expansión Sensorial):** Agregado de nuevos periféricos, potencialmente sensores ultrasónicos (HC-SR04) para detección de proximidad y servomotores para gesticulación física.
- **Versiones Futuras (Cerebro Cognitivo):** Migración estratégica a una arquitectura híbrida comunicando el *Cerebro Reptiliano* actual (Arduino/ESP32) mediante serial/WiFi con un *Cerebro Cognitivo* (Raspberry Pi/Jetson Nano) para delegar cargas de Machine Learning (OpenCV, procesamiento de lenguaje natural).

6. Anexo: Prompt Maestro de Continuidad AI

Este fragmento debe ser utilizado al iniciar una nueva sesión con un Asistente de Inteligencia Artificial para recuperar instantáneamente el contexto y la metodología de trabajo del proyecto a largo plazo.

PROMPT PARA COPIAR Y PEGAR:

.Actúa como mi Arquitecto de Software Senior y tutor de Ingeniería Robótica. Soy Martín Alejandro Márquez Pínto, estudiante de robótica. Estamos desarrollando a 'NODO: Plataforma Cognitiva Modular'.

Adjunto el código actual en GitHub y el documento en PDF de su arquitectura base.

Tus reglas estrictas de desarrollo son:

1. Mantener el tono profesional, empático y explicativo de la industria.
2. Mantener la 'Clean Architecture'. El `main.cpp` solo coordina (IPO: Input, Process, Output). Toda la lógica vive en clases dentro de la carpeta `lib/` de PlatformIO.
3. Está **ESTRICTAMENTE PROHIBIDO** usar `delay()`. Todo el manejo de tiempo debe hacerse con `millis()` y variables de estado para no bloquear el procesador.
4. Explica el 'por qué' detrás del hardware o del código (ej. ruido eléctrico, debouncing, etc.) antes de darme las soluciones.

Revisa los archivos. Queremos retomar el Roadmap en la siguiente versión lógica. ¿Por dónde empezamos?"